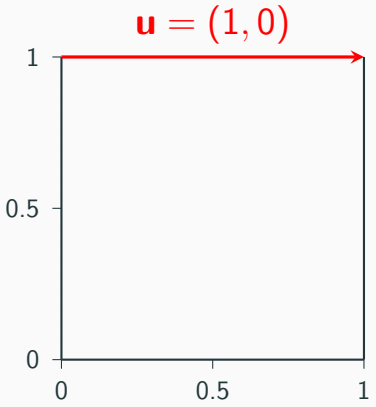
Simulating lid-driven cavity

Lukas Lanik

October 22, 2026

The problem at hand

Simulating the flow of a viscuous fluid in a 2D cavity



Stokes problem

$$\begin{cases} -\Delta \mathbf{u} + \nabla p = 0, & \mathbf{x} \in \Omega \\ \operatorname{div} \mathbf{u} = 0, & \mathbf{x} \in \Omega \end{cases}$$

$$\mathbf{u} = \mathbf{0}, \qquad \mathbf{x} \in \partial\Omega \setminus \Gamma,$$

$$\mathbf{u} = (1, 0)^T, \qquad \mathbf{x} \in \Gamma,$$

where $\Gamma = \{\mathbf{x} = (x_1, x_2)^T \in \partial\Omega : x_2 = 1\}$

Solution methods

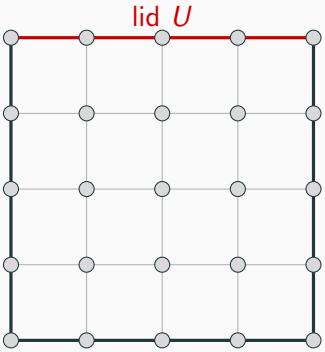
- Finite difference/ Staggered MAC
- Spectral methods
- Finite Element Method

Solution methods

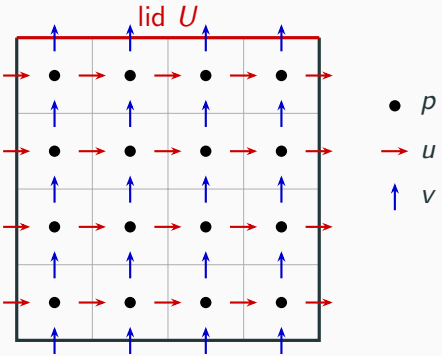
All of the methods create a system

$$\begin{pmatrix} A & B^T \\ B & 0 \end{pmatrix} \begin{pmatrix} \mathbf{U} \\ \mathbf{P} \end{pmatrix} = \begin{pmatrix} \mathbf{f} \\ \mathbf{g} \end{pmatrix}$$

Finite difference/ Staggered MAC



Collocated: u, v, p at nodes



MAC: p at centres, u, v on faces

Finite difference

$$
\begin{aligned}
-(\Delta_h u_1)_{i,j} + \frac{p_{i+1,j} - p_{i-1,j}}{2h} &= 0, \\
-(\Delta_h u_2)_{i,j} + \frac{p_{i,j+1} - p_{i,j-1}}{2h} &= 0, \\
\frac{(u_1)_{i+1,j} - (u_1)_{i-1,j}}{2h} + \frac{(u_2)_{i,j+1} - (u_2)_{i,j-1}}{2h} &= 0,
\end{aligned}
$$

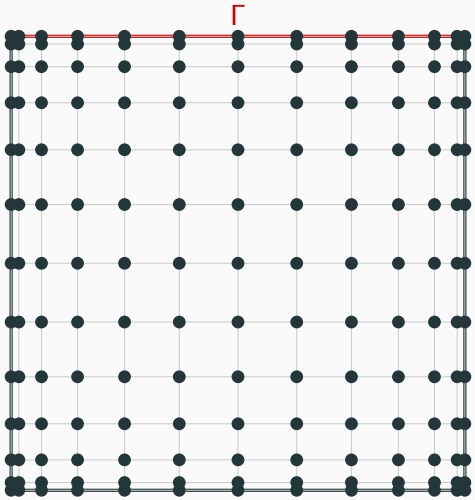
with $\mathbf{u}_{i,j} = (1,0)^T$ on Γ and $\mathbf{u}_{i,j} = \mathbf{0}$ on $\partial\Omega \setminus \Gamma$.

Staggered MAC

$$
\begin{aligned}
-(\Delta_h u_1)_{i+\frac{1}{2},j} + \frac{p_{i+1,j} - p_{i,j}}{h} &= 0, \\
-(\Delta_h u_2)_{i,j+\frac{1}{2}} + \frac{p_{i,j+1} - p_{i,j}}{h} &= 0, \\
\frac{(u_1)_{i+\frac{1}{2},j} - (u_1)_{i-\frac{1}{2},j}}{h} + \frac{(u_2)_{i,j+\frac{1}{2}} - (u_2)_{i,j-\frac{1}{2}}}{h} &= 0,
\end{aligned}
$$

with ghost values $(u_1)_{\text{ghost}} = 2 - (u_1)_{\text{interior}}$ on Γ.

Spectral Methods



Chebyshev–Gauss–Lobatto points, $N = 12$

Spectral Methods

$$u_1^N(\mathbf{x}) = \sum_{k,l=0}^{N} \hat{u}_{kl}\, T_k(x_1)\, T_l(x_2), \qquad p^N(\mathbf{x}) = \sum_{k,l=0}^{N-2} \hat{p}_{kl}\, T_k(x_1)\, T_l(x_2),$$

similarly for u_2^N, with $T_k(\cos\theta) = \cos(k\theta)$.

Finite Element Method

Let $\mathbf{v} \in V$ and $q \in Q$, find $\mathbf{u} \in V$, $p \in Q$ such that

$$V = \{\mathbf{v} \in \left[\mathrm{H}^1(\Omega)\right]^2 : \mathbf{v}|_{\Gamma_D} = 0\}$$

$$Q = \{q \in \mathrm{L}^2(\Omega) : \int_\Omega q \, d\Omega = 0\}$$

$$\int_\Omega \nabla \mathbf{u} \cdot \nabla \mathbf{v} \, d\Omega - \int_\Omega p \, \mathrm{div}\, \mathbf{v} \, d\Omega = 0$$

$$\int_\Omega (\mathrm{div}\, \mathbf{u}) \, q \, d\Omega = 0$$

Galerkin approximation

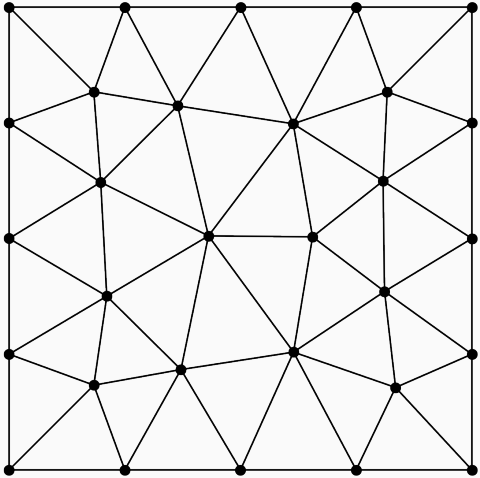
Let $V_h \subset V$, $Q_h \subset Q$ be finite dimensional subspaces. Find $u_h = \sum_{i=0}^{N_h} u_i \varphi_i$, $p_h = \sum_{k=0}^{M_h} p_k \phi_k$.

The Stokes problem becomes

$$\begin{cases} \mathrm{A}\mathbf{U} + \mathrm{B}^T\mathbf{P} = 0 \\ \mathrm{B}\mathbf{U} = 0 \end{cases}$$

$$\mathrm{A} = [a_{ij}] = [a(\varphi_j, \varphi_i)], \quad \mathrm{B} = [b_{km}] = [b(\varphi_m, \phi_k)]$$

Meshing



- choose target mesh size h (size of an edge)
- non-overlapping
- unstructured

FEniCSx

- computation of PDEs
- DOLFINX, UFL, FFCx, Basix
- variational form $\rightarrow$ matrix
- C++ with python bindings

Code setup

```
from dolfinx import fem

V = fem.functionspace(mesh, ("Lagrange", 2, (2,)))
Q = fem.functionspace(mesh, ("Lagrange", 1))

u = ufl.TrialFunction(V)
v = ufl.TestFunction(V)
p = ufl.TrialFunction(Q)
q = ufl.TestFunction(Q)
```

Code setup

```
a00 = ufl.inner(ufl.grad(u), ufl.grad(v)) * ufl.dx
a01 = -p * ufl.div(v) * ufl.dx
a10 = -q * ufl.div(u) * ufl.dx
a11 = None

L0 = ufl.inner(f0, v) * ufl.dx
L1 = f1 * q * ufl.dx
```

Boundary conditions

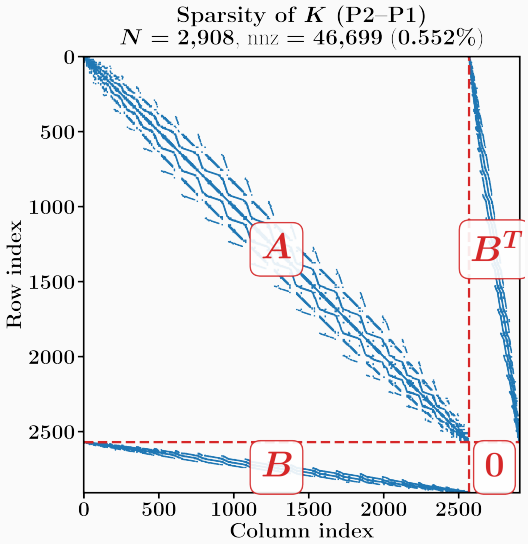
Assemble the full system first:

$$\begin{bmatrix} A_{ff} & A_{fb} \\ A_{bf} & A_{bb} \end{bmatrix} \begin{bmatrix} u_f \\ u_b \end{bmatrix} = \begin{bmatrix} b_f \\ b_b \end{bmatrix}$$

afterwards apply boundary conditions:

$$\Rightarrow \begin{bmatrix} A_{ff} & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} u_f \\ u_b \end{bmatrix} = \begin{bmatrix} b_f - A_{fb} u_D \\ u_D \end{bmatrix}$$

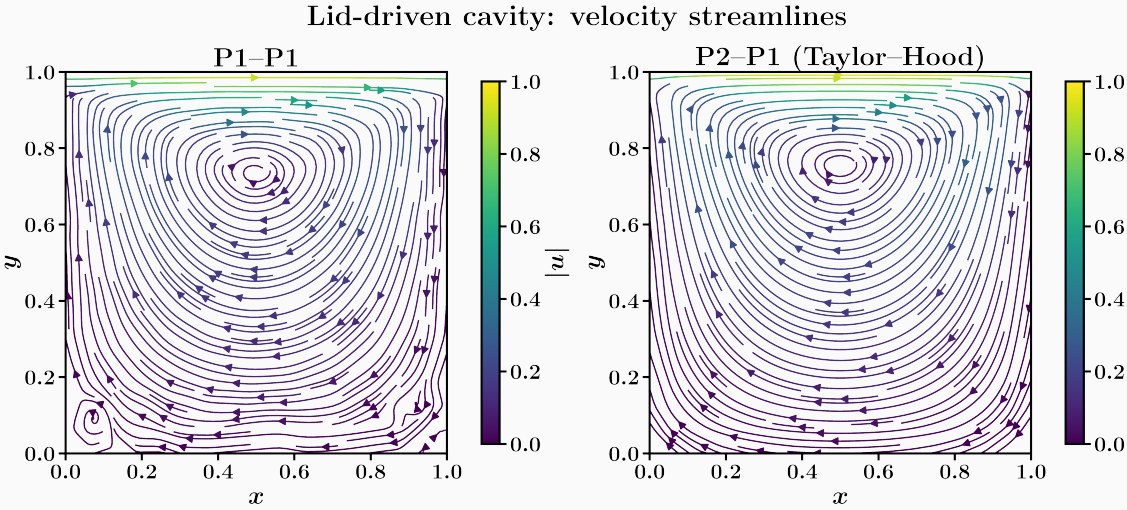
Matrix assembly



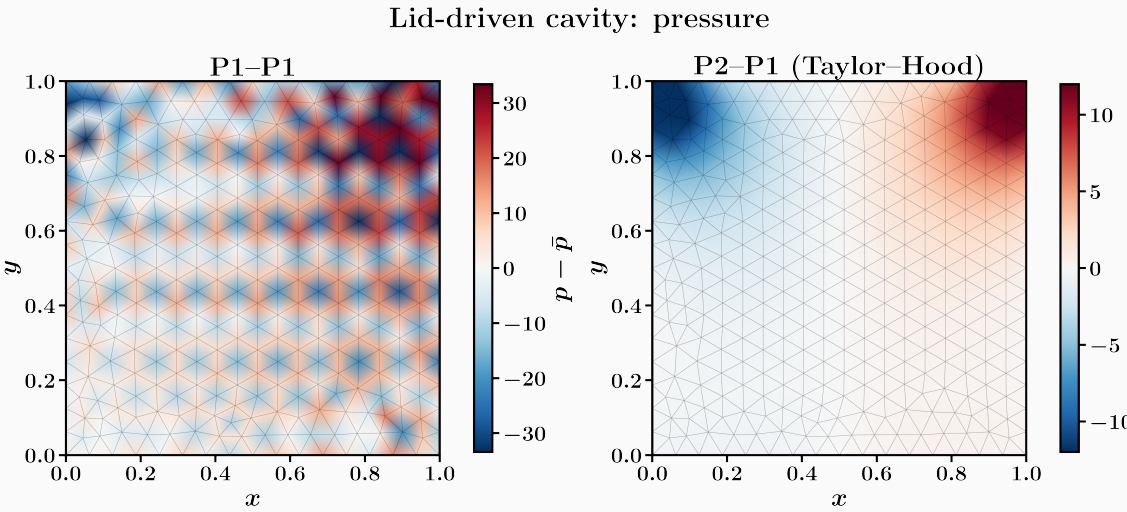
Possible basis elements

- FEM elements library: defelement.org
- 67 different elements listed
- ones used for investigation: P2-P1, P1-P1, P1b-P1

Stable and unstable solutions



Stable and unstable solutions



Code validation - manufactured solutions

Take a known solution and plug it into existing code

$$\mathbf{u}(\mathbf{x}) = \pi \begin{pmatrix} \sin(\pi x_1)\cos(\pi x_2) \\ -\cos(\pi x_1)\sin(\pi x_2) \end{pmatrix}, \qquad p(\mathbf{x}) = \cos(\pi x_1)\cos(\pi x_2).$$

derived from $\psi(\mathbf{x}) = \sin(\pi x_1)\sin(\pi x_2)$ via $\mathbf{u} = (\partial_{x_2}\psi,\ -\partial_{x_1}\psi)^T$, so $\operatorname{div}\mathbf{u} = 0$ and $\int_\Omega p = 0$.

A priori error estimates

1. Quasi-optimality (inf-sup stability)

$$|\mathbf{u} - \mathbf{u}_h|_{H^1} + \|p - p_h\|_{L^2} \leq C(\beta)\Big(\inf_{\mathbf{v}_h \in V_h} |\mathbf{u} - \mathbf{v}_h|_{H^1} + \inf_{q_h \in Q_h} \|p - q_h\|_{L^2} \Big)$$

2. Approximation (Bramble–Hilbert, shape regularity)

$$|\mathbf{u} - I_h\mathbf{u}|_{H^1} \leq Ch^k |\mathbf{u}|_{H^{k+1}}, \qquad \|p - I_h p\|_{L^2} \leq Ch^k |p|_{H^k}$$

3. Duality (Aubin–Nitsche, Ω convex)

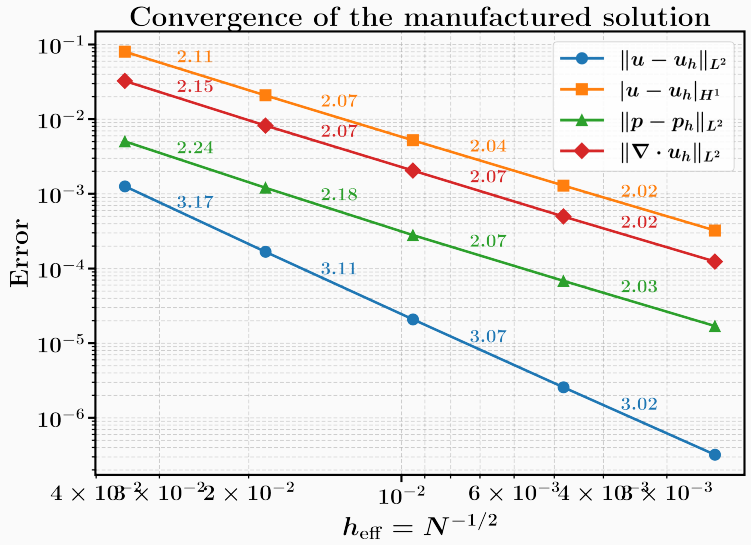
$$\|\mathbf{u} - \mathbf{u}_h\|_{L^2} \leq Ch \left(|\mathbf{u} - \mathbf{u}_h|_{H^1} + \|p - p_h\|_{L^2} \right)$$

Expected convergence rates: Taylor–Hood P_2/P_1

$$|\mathbf{u} - \mathbf{u}_h|_{H^1} + \|p - p_h\|_{L^2} \leq Ch^2, \qquad \|\mathbf{u} - \mathbf{u}_h\|_{L^2} \leq Ch^3$$

Error	Expected rate	Comes from
$\|\mathbf{u} - \mathbf{u}_h\|_{H^1}$	2	steps 1 + 2
$\|p - p_h\|_{L^2}$	2	steps 1 + 2
$\|\mathbf{u} - \mathbf{u}_h\|_{L^2}$	3	step 3

Expected convergence rates: Taylor–Hood P_2/P_1



PETSc

- scientific computation toolkit
- linear and nonlinear equation solvers
- ODE integrators
- optimization algorithms
- C, Python, Fortran
- GPU integration and parallelism

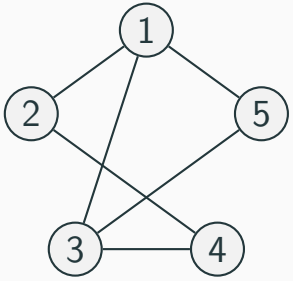
Linear solver algorithms - LU factorisation

Elimination creates new connections creating fill-in

Symmetric Matrix A

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & 0 & a_{15} \\ a_{21} & a_{22} & 0 & a_{24} & 0 \\ a_{31} & 0 & a_{33} & a_{34} & a_{35} \\ 0 & a_{42} & a_{43} & a_{44} & 0 \\ a_{51} & 0 & a_{53} & 0 & a_{55} \end{bmatrix}$$

Undirected Graph $\mathcal{G}(A)$



Linear solver algorithms - Krylov methods

Iteratively solve $A\mathbf{x} = \mathbf{b}$ such that kth iterate $\mathbf{x}^{(k)}$ lies in the Krylov space:

$$\mathbf{x}^{(0)} + \text{span}\{\mathbf{r}^{(0)}, A\mathbf{r}^{(0)}, A^2\mathbf{r}^{(0)}, \cdots, A^{k-1}\mathbf{r}^{(0)}\}$$

where $\mathbf{r}^{(0)} = \mathbf{b} - A\mathbf{x}^{(0)}$.

MINRES

$$\mathbf{x}^{(k)} = \underset{\mathbf{x} \in \mathbf{x}^{(0)} + \mathcal{K}_k}{\arg\min} \|\mathbf{b} - A\mathbf{x}\|_2$$

- symmetric indefinite systems
- short recurrence (Lanczos)
- preconditioner $\mathcal{P} = \begin{pmatrix} \hat{A} & 0 \\ 0 & \hat{M}_p \end{pmatrix}$

Preconditioning

Solving $A\mathbf{x} = \mathbf{b}$ may be ill-conditioned. Hence apply P^{-1}, thus the system becomes

$$P^{-1}A\mathbf{x} = P^{-1}\mathbf{b}.$$

Choose P such that

- eigenvalues of $P^{-1}A$ are clustered
- $P^{-1}\mathbf{v}$ is cheap to compute
- P is SPD (required by MINRES)

Field-split preconditioning

$$\mathcal{A} = \begin{pmatrix} A & B^T \\ B & 0 \end{pmatrix}, \qquad \mathcal{P} = \begin{pmatrix} \hat{A} & 0 \\ 0 & \hat{M}_p \end{pmatrix}$$

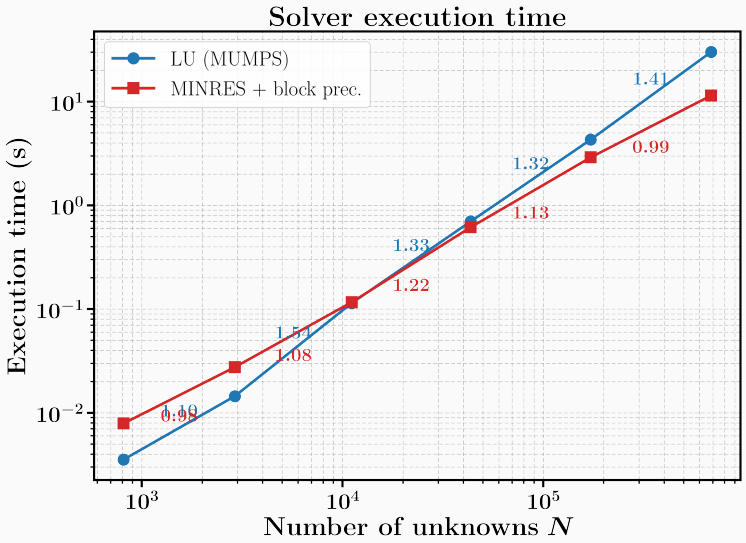
`additive`	block diagonal
$\hat{A}$	one AMG V-cycle (`gamg`)
$\hat{M}_p$	diag(M_p) (`jacobi`)

Expected computational cost

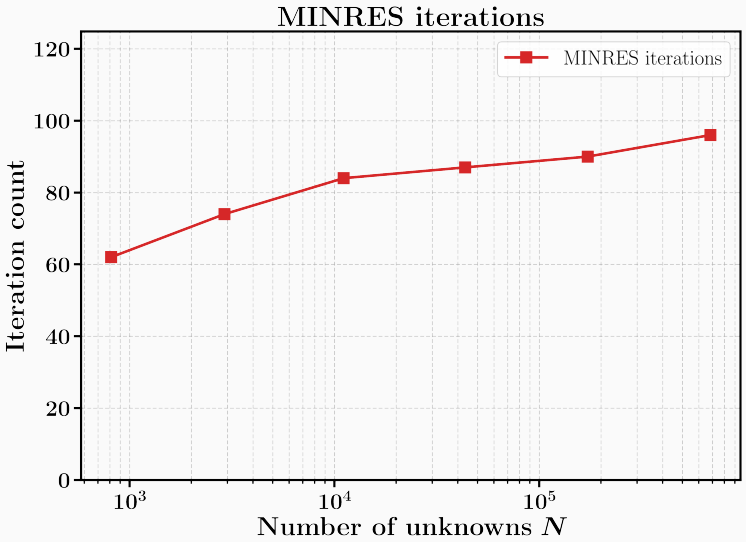
Solver	Time	Memory
LU (MUMPS)	$O(n^{3/2})$	$O(n \log n)$
MINRES + field-split	$O(n)$	$O(n)$

slope in $\log T$ vs. $\log n$: LU ≈ 1.5, MINRES ≈ 1

Solver algorithms compared



Solver algorithms compared



Reflections on AI use

1. General topic research: valuable
2. Brainstorming, reasoning check: great
3. Coding simple scripts: efficient
4. Coding niche requirements: unsatisfactory

Thank you for your attention!